Phase 5 — vLLM Baseline & Quantization Study (GPU)

Goal of this phase: bring in vLLM as the *production* baseline to benchmark my engine against honestly, and study the FP16-vs-INT8 quantization tradeoff (speed/memory vs quality). Everything that can be prepared on CPU is done; this doc is also the **GPU runbook** so the paid session is short.

> Status: **complete.** All three engines (naive, batched, vLLM) benchmarked on the **same NVIDIA L4** (GCP spot VM) through one harness; FP16/INT8/INT4 quantization study measured. Results below.

---

0. Measured results — three engines on one NVIDIA L4 (Qwen2.5-0.5B, 128-token generations)

All three engines were benchmarked on the **same L4** through the same load harness (spot VM on GCP).

Throughput (tokens/sec, aggregate):

engine	c=1	c=4	c=16	c=64
naive	32.0	35.0	34.5	34.9
batched (mine)	39.6	119.1	214.4	213.0
vLLM (PagedAttention)	48.3	177.5	**655.1**	**2254.5**

p99 latency (s): naive 7.3 → 8.6 → 37.0 → **135.9** (explodes); batched 3.3 → 4.1 → 7.5 → 18.7; vLLM 2.7 → 2.9 → 3.0 → 3.4 (near-flat). **Peak GPU memory:** naive/batched ~1.3 GB, vLLM ~21.5 GB.

The three-way story (the whole point of the project): - **naive → batched:** my continuous-batching engine is **~6× naive** at c=16 (214 vs 34 tok/s) and keeps p99 bounded (19s vs naive's 136s at c=64). Naive serializes, so throughput is flat and the tail explodes. - **batched → vLLM:** vLLM is **~3× my engine at c=16 and ~10× at c=64** (2255 vs 213 tok/s). vLLM's **PagedAttention** stores the KV cache in non-contiguous fixed-size pages, so it packs far more concurrent sequences into VRAM and keeps scaling where my **contiguous left-padded cache** saturates. vLLM pre-reserves ~21.5 GB for its paged pool (vs my 1.3 GB) — it trades memory for that packing. My engine is the from-scratch continuous-batching idea; vLLM is the paged, fragmentation-free version.

Honest measurement caveats: vLLM is driven via OpenAI `/v1/completions` on the raw prompt (no chat template), so it generated ~121 tok/req vs my ~63 — tok/s normalizes this but the workloads aren't byte-identical; and vLLM's "TTFT" here is full request latency (the harness doesn't stream vLLM), so only naive-vs-batched TTFT is directly comparable.

Quantization (bitsandbytes; measured on a T4 in an earlier run):

precision	throughput (tok/s)	peak mem (MiB)	perplexity	agreement vs FP16
FP16	27.8	979.7	14.04	1.00
INT8	6.9	639.7	14.22	0.64
INT4 (NF4)	17.7	489.3	25.61	0.07

Finding (honest, non-obvious): on a 0.5B model, quantization **saved memory but not time** — INT8/INT4 were *slower* than FP16 because bitsandbytes' dequant overhead dominates at this scale, and INT4 quality collapsed (perplexity 14→26, agreement 1.00→0.07). Quantization's *throughput* payoff needs large models or hardware with **native INT8/FP8 kernels**; its *memory* payoff (980→489 MiB) is real and immediate. Sharper than the usual "quantization = faster," which is often false.

---

1. What vLLM does that my engine doesn't — PagedAttention

My engine stores each sequence's KV cache as **one contiguous, left-padded block**. Two costs: - **Padding waste:** every sequence is padded to the batch's max length; padded slots burn memory and compute. - **Copying / fragmentation:** admitting and evicting sequences shuffles cache memory, and reserving a contiguous block per sequence fragments the GPU.

PagedAttention (vLLM's core idea) treats the KV cache like an OS treats virtual memory: - The cache is split into fixed-size **blocks (pages)**; a sequence's tokens are stored in possibly-non-contiguous pages, tracked by a **block table** (an indirection layer). - **No padding** — a sequence uses exactly as many pages as it needs, growing one page at a time. - **Near-zero fragmentation** — any free page can serve any sequence, so memory utilization is high and far more sequences fit in the same VRAM → bigger batches → higher throughput. - **Copy-on-write sharing** — shared prefixes (e.g. a common system prompt) can share pages.

vLLM also does **continuous batching** (like mine) plus **chunked prefill** (interleaving prefill and decode so prefill bursts don't stall decode / spike TTFT — the exact weakness Phase 4 found in my engine). Net: vLLM should clearly out-throughput my engine, and I can say *precisely* why — it's the paged, fragmentation-free version of the cache I built by hand.

Honest framing for interviews: "My engine implements continuous batching with a contiguous padded cache. vLLM implements the same idea with a paged cache, so it packs more concurrent sequences into memory and protects TTFT with chunked prefill. I expect to land within some percentage of its throughput and lose most on high-concurrency, long-context workloads where paging matters most."

---

2. What I built (prepared on CPU)

- **Same-harness vLLM benchmarking:** the Phase 4 load tester now speaks the **OpenAI completions API** (`--api openai`), so I point the identical harness at vLLM's built-in server and get comparable throughput/latency numbers. (Validated on CPU against a fake OpenAI server in tests.)
- **`engine/quant.py`** — FP16 / INT8 / INT4(NF4) model loaders via `bitsandbytes` (GPU-only).
- **`benchmark/quality.py`** — a defensible quality metric: **perplexity** on a fixed coherent-text eval set (lower = better) plus **token-agreement vs FP16** (how often greedy decoding matches the full-precision reference). Validated on CPU on the real model.
- **`benchmark/quant_compare.py`** — GPU script: for each precision, measures decode throughput, peak GPU memory, perplexity, and FP16-agreement, and writes `results/quant.json`.
- **`requirements-gpu.txt`** — `vllm`, `bitsandbytes` (installed only on the GPU box).

Verification (CPU): 20/20 tests pass, including perplexity sanity and the OpenAI adapter.

---

3. Why perplexity + token-agreement as the quality metric

Speed/memory are easy to measure; "did quantization hurt quality?" needs a number. My two: - **Perplexity** on fixed fluent text — model-agnostic, standard, needs no labels. If INT8 perplexity is ~equal to FP16, quantization didn't meaningfully degrade the model. - **Token-agreement vs FP16** — with greedy decoding, what fraction of generated tokens are identical to the FP16 reference? Directly answers "does the quantized model behave like the original?"

Together they show the **tradeoff, not just speed**: e.g. "INT8 cut memory ~2× and matched FP16 perplexity within 1% with 95%+ token agreement — a free win; INT4 saved more memory but perplexity rose and agreement dropped, so it's only worth it under memory pressure."

---

4. GPU RUNBOOK (do this in one focused, budget-aware session)

Cheapest viable setup: one **spot/preemptible** GPU VM (e.g. L4 or T4 — plenty for a 0.5B model), single GPU. Set a **$200 budget alert** first. (Full VM creation steps are in Phase 6.)

Step 0 — confirm you're on the GPU (don't skip):

```
nvidia-smi                                    # must list a GPU
python -c "import torch; print(torch.cuda.is_available())"   # must print True
```

Step 1 — install GPU deps:

```
pip install -r requirements.txt -r requirements-gpu.txt
```

Step 2 — benchmark MY engines (naive + batched), native API:

```
python benchmark/runner.py --concurrency 1,4,16,64 --requests 64 --max-tokens 128
mv results/latest.json results/mine.json     # keep my numbers
```

Step 3 — benchmark vLLM, same harness (in a second terminal start vLLM):

```
# terminal A: start vLLM's OpenAI server
vllm serve Qwen/Qwen2.5-0.5B-Instruct --port 8000
# terminal B: point the SAME harness at it
python benchmark/runner.py --url http://127.0.0.1:8000 --api openai \
  --model Qwen/Qwen2.5-0.5B-Instruct --engines vllm \
  --concurrency 1,4,16,64 --requests 64 --max-tokens 128
mv results/latest.json results/vllm.json
```

Step 4 — quantization study (FP16 vs INT8 vs INT4):

```
python benchmark/quant_compare.py fp16,int8,int4   # writes results/quant.json
```

Step 5 — sanity-check GPU was actually used: while steps 2–4 run, a second `watch -n1 nvidia-smi` should show utilization >0 and memory in use. The harness also records `util_mean_pct` / `mem_max_mib`.

Step 6 — SHUT DOWN (impossible to forget):

```
# copy results off the box first
# then STOP/DELETE the VM and confirm in the billing console nothing is running
```

Full teardown checklist is in Phase 6. **A100 ≈ $3–4/hr — never leave it idle.**

---

5. Interview Q&A

Q: What is PagedAttention and why is it faster? A: It stores the KV cache in fixed-size pages with a per-sequence block table, like OS virtual memory. That removes padding and fragmentation, so far more concurrent sequences fit in the same VRAM — bigger effective batches and higher throughput. It also enables prefix sharing via copy-on-write pages.

Q: How is vLLM different from your continuous-batching engine? A: Same scheduling idea (admit/evict in-flight), different memory model. I use one contiguous padded cache block per sequence; vLLM uses paged, non-contiguous cache with an indirection table. vLLM adds chunked prefill to protect TTFT. So it packs more sequences and avoids the TTFT spike my engine showed under burst.

Q: Why might your engine still be within X% of vLLM on small workloads? A: At low concurrency and short context, padding/fragmentation barely matter and a single batched forward is a single batched forward. Paging wins as concurrency and context length grow — that's where the gap should open up, and my graphs should show exactly that.

Q: How do you measure quantization quality, not just speed? A: Perplexity on fixed fluent text (lower is better, no labels needed) and token-agreement vs the FP16 output under greedy decoding. That turns "is it still good?" into two numbers, so I can state the tradeoff: memory/throughput gained vs quality lost.

Q: FP16 vs INT8 vs INT4 — when would you ship each? A: FP16 when memory isn't the bottleneck and you want reference quality. INT8 is usually the sweet spot — roughly half the memory with near-identical quality. INT4 only under real memory pressure (fit a bigger model or more concurrency), accepting measurable quality loss. The quant table makes that call data-driven.

Q: Why benchmark vLLM through its own server instead of importing it? A: vLLM's OpenAI server is the production-standard path and what people actually deploy. Reusing my HTTP harness against it gives an apples-to-apples comparison with my engines and tests the real serving stack, not a synthetic in-process call.

---

6. One-line recall

> *"I benchmarked vLLM against my own engine through the same load harness (via its OpenAI server), and ran an FP16/INT8/INT4 study measuring throughput, GPU memory, and quality (perplexity + FP16-agreement). I can explain exactly where vLLM pulls ahead — PagedAttention's paged, padding-free KV cache and chunked prefill — because I built the contiguous-cache version it improves on."*